DocuSaathi

Government document-to-action assistant, built on Sarvam AI

Hackathon track	Applied AI
Problem statement	PS 41: use at least one Sarvam API to solve a problem that matters
Duration	24 hours
One-line pitch	Photograph any confusing official document and get, in your own language and by voice, what it is, what you must do, and by when.

1. The problem

Official notices, bank letters, scheme forms and land records are dense and usually written in formal English or bureaucratic language. Millions of people in India cannot easily read them, so they miss deadlines, skip benefits or depend on middlemen. Most AI tools are built for English speakers, which leaves exactly these users behind.

2. The solution

DocuSaathi converts a photo or PDF of an official document into an action plan. It does not just translate. It answers three questions:

- **What is this?** The document type and who sent it.
- **What do I have to do?** A prioritized checklist of actions.
- **By when, and what happens if I don't?** Deadlines, amounts and consequences.

The answer is given in the user's language as text and as voice, and the user can ask follow-up questions by speaking.

3. Target users

- Elderly citizens and first-generation literate users who struggle with formal English.
- Rural and semi-urban users who prefer voice over typing.
- Small business owners handling tax, GST and bank notices.
- Family members and community workers helping others with paperwork.

4. End-to-end workflow

Seven stages run in sequence. Each stage has a clear input and output, so team members can build and test them in parallel.

#	Stage	Input	Output
1	Input	Photo / PDF, language choice	Image or PDF in a session
2	Document reading	Image / PDF	Clean text with layout and page regions
3	Classification and extraction	Text	Structured JSON with confidence
4	Action engine	Structured JSON	Prioritized checklist
5	Plain-language explanation	Checklist and JSON	Simple explanation in user's language
6	Voice layer	Spoken question	Grounded spoken answer
7	Output and handoff	All results	Card view, PDF / WhatsApp-ready share, reminders

5. Stage-by-stage detail

STAGE 1	Input
What happens	The user uploads a photo or PDF, or forwards one from the phone. They choose a language, or the system detects it from their first voice question. A quality check warns about blur, glare or cut-off edges and asks for a retake.
Tech used	Web app (mobile first), image preprocessing (rotate, deskew, contrast).
Output	A clean document image tied to a session ID.

STAGE 2	Document reading
What happens	Extracts text, tables, stamps and layout from the scan. Handles low-quality phone photos and pages that mix English with a regional language. Keeps page and region references for later highlighting.
Tech used	Sarvam Vision (document intelligence).
Output	Clean text plus coordinates for each block.

STAGE 3	Classification and extraction
What happens	Identifies the document type (tax notice, scheme application, bank KYC letter, legal notice, utility bill, etc.) and fills a fixed schema. Each field carries a confidence level and the source snippet it came from.
Tech used	Sarvam chat model with a strict JSON schema prompt, plus code-based validation (regex for dates, amounts, IDs).
Output	Structured JSON (see schema in section 6).

STAGE 4	Action engine
What happens	Converts extracted fields into a checklist: what to do (pay, reply, visit, submit), the deadline and consequence, documents to carry, and where to go. Ranks items by urgency. Rule-based checks run alongside the LLM, for example marking a past deadline as overdue.
Tech used	Sarvam chat model for reasoning, deterministic rules in code for dates and priorities.
Output	Ordered checklist with urgency labels.

STAGE 5	Plain-language explanation
What happens	Rewrites the content at a simple reading level in the user's language. Official terms are kept with a short meaning next to them so the user can still use them at the office.
Tech used	Sarvam chat model, Sarvam translation and transliteration for script handling.
Output	Short explanation and checklist in the chosen language.

STAGE 6	Voice layer
What happens	The user asks follow-ups by voice, for example "Can I pay this online?" or "What happens if I am late?". The answer is generated only from the document content, with a clear "not mentioned in this document" fallback, and is spoken back.
Tech used	Saaras (speech-to-text), Sarvam chat model with document-grounded prompting, Bulbul (text-to-speech).
Output	Spoken and written answer in the user's language.

STAGE 7	Output and handoff
What happens	Shows a card-style summary: document type, deadline, steps, documents. The user can share it as a WhatsApp-ready message or PDF with a family member. Optional deadline reminders. Documents are deleted after the session.
Tech used	Frontend rendering, PDF export, share links.
Output	Summary card, shareable file, optional reminder.

6. Data schema (the backbone)

Every stage after reading works on this schema. Agreeing on it in the first two hours lets the whole team build in parallel.

```
{
  "doc_type": "income_tax_notice",
  "issuer": "...",
  "key_dates": [
    {"label": "Reply by", "date": "...", "source": "page1 line4"}
  ],
  "amounts": [
    {"label": "Amount due", "value": "...", "source": "..."}
  ],
  "obligations": ["Respond to notice", "Pay difference"],
  "required_documents": ["PAN copy", "..."],
  "consequence_if_ignored": "...",
  "where_to_act": {"portal": "...", "office": "..."},
  "confidence": {"overall": 0.0, "low_fields": []}
}
```

7. System layout

Layer	Responsibility
Frontend	Mobile-friendly web app: upload, language selector, mic button, checklist view, highlighted original document.
Backend API	Pipeline orchestration (Vision, extraction, action engine, explanation, TTS) and session storage so follow-up questions keep context.
Sarvam services	Vision for reading, chat model for extraction and reasoning, Saaras for speech-to-text, Bulbul for text-to-speech, translation APIs.
Validation module	Code checks for dates, amounts and ID formats against the extracted text, to catch hallucinations.
Evaluation script	10 to 15 realistic documents with expected fields, producing accuracy numbers for the demo.

8. Trust and safety

- **Evidence:** every extracted field links to the highlighted region of the original document.
- **Confidence flags:** blurry scans or uncertain fields are marked instead of guessed.
- **Grounded answers:** follow-up answers use only the document, with a not-mentioned fallback.
- **Not legal advice:** the app explains documents and points users to the official office or helpline for decisions.
- **Privacy:** documents are not stored beyond the session and are deleted afterward; this is stated in the pitch.

9. 24-hour build plan

Hours	Work
0-2	Lock scope, get API keys, finalize the JSON schema, collect 10 to 15 sample documents (include blurry phone photos).
2-10	Backend pipeline (reading, extraction, action engine, explanation) in parallel with the frontend shell and share/export.
10-16	Voice layer, evidence highlighting, second language, validation checks, error handling.
16-20	Integration, run the evaluation set, fix the weakest outputs.
20-24	Demo script, slides, backup recorded demo, README.

10. Scope

Must have	Nice to have	Skip
Upload to checklist to explanation in 2 languages Voice question and spoken answer Evidence highlighting on the original	WhatsApp-style share Deadline reminders Third language	User accounts Real government integrations Payments

11. Demo script (about 2 minutes)

- Show a dense English notice and explain who gets confused by it.
- Upload a phone photo and watch the checklist appear in the chosen language.
- Tap a requirement and show it highlighted on the original page.
- Ask a voice question in the local language and play the spoken answer.
- Close with the accuracy numbers from the test set.

12. Risks and mitigations

Risk	Mitigation
Poor OCR on bad photos	Test early with real phone pictures; add a retake prompt and preprocessing.
Hallucinated dates or amounts	Validate with code against the extracted text; show confidence flags.
Sensitive personal data	No storage beyond the session; delete after use; state this clearly.
Scope creep	Freeze features at hour 16; polish one flow instead of adding more.
Voice latency in demo	Pre-test audio, keep answers short, have a recorded backup.

13. Judging angles to highlight

- Clear user and real-world problem: citizens who cannot read official documents.
- Sarvam used where it matters: vision, speech, language and voice together.
- Measurable quality: accuracy on a test set, not just a nice demo.
- Responsible design: evidence, confidence flags, grounded answers, privacy.